Lab 8: Task Scheduling Project

Lab Exercises and Modular Scheduling Simulator

Santiago Soto
CS 375 – Operating Systems

October 13, 2026

GitHub repository:
`https://github.com/stjamessoto/Module-4-Assignment-3-Lab-Scheduling-Project`

Contents

1 Overview **2**

2 Part 2: Lab Exercises **2**
2.1 Exercise 1: FCFS (`fcfs.cpp`) . . . 2
2.2 Exercise 2: SJF, non-preemptive (`sjf.cpp`) . . . 3
2.3 Exercise 3: SRTF, preemptive (`srtf.cpp`) . . . 4
2.4 Exercise 4: Priority with aging (`priority.cpp`) . . . 5
2.5 Exercise 5: Round Robin (`rr.cpp`) . . . 6
2.6 Exercise 6: Multilevel Queue (`mlq.cpp`) . . . 7
2.7 Exercise 7: Multilevel Feedback Queue (`mlfq.cpp`) . . . 8
2.8 Exercise 8: Lottery (`lottery.cpp`) . . . 9
2.9 Exercise 9: Simplified CFS (`cfs.cpp`) . . . 10
2.10 Exercise 10: EDF (`edf.cpp`) . . . 11

3 Part 3: Modular Task Scheduling Simulator **12**
3.1 Design . . . 12
3.2 Usage . . . 13
3.3 Enhancements . . . 13
3.4 Test Runs . . . 14
3.5 Comparison and Analysis . . . 20

4 References **22**

1 Overview

This report covers both parts of the lab. Part 2 is ten standalone C++ programs, one per scheduling algorithm, each run on the process table from the assignment (Table 1). Part 3 combines the same ten algorithms into one command-line simulator built around a polymorphic `Scheduler` base class, and uses it to compare the schedulers on larger workloads.

Process	Arrival Time	Burst Time	Priority (lower = higher)
P1	0	8	2
P2	1	4	1
P3	2	9	3
P4	3	5	4

Table 1: Process table used for every lab exercise.

Every program prints the process table, a text Gantt chart in the form `ID(duration)` with a timeline underneath, the waiting and turnaround time of each process, and the average waiting time, average turnaround time and CPU utilization. CPU utilization is computed as total burst time divided by elapsed time, so idle time is counted.

2 Part 2: Lab Exercises

Table 2 summarises the results of the ten programs on the assignment table. All ten finish at $t = 26$ with 100% CPU utilization, because the CPU is never idle.

Ex.	Scheduler	Avg waiting	Avg turnaround
1	FCFS	8.75	15.25
2	SJF (non-preemptive)	7.75	14.25
3	SRTF (preemptive)	6.50	13.00
4	Priority with aging	8.75	15.25
5	Round Robin ($q = 4$)	11.75	18.25
6	Multilevel Queue	8.75	15.25
7	Multilevel Feedback Queue	14.00	20.50
8	Lottery	varies with the random seed	
9	CFS (simplified)	12.25	18.75
10	EDF	6.50	13.00

Table 2: Lab results on the assignment process table.

2.1 Exercise 1: FCFS (`fcfs.cpp`)

Processes are sorted by arrival time and each runs to completion. If the next process has not arrived yet, the CPU idles until it does. The order is P1, P2, P3, P4.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/fcfs
=== Exercise 1: First-Come, First-Served (FCFS) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Gantt Chart: P1(8) P2(4) P3(9) P4(5)
Timeline:    0 [P1] 8 [P2] 12 [P3] 21 [P4] 26

Process  Waiting  Turnaround
P1       0        8
P2       7        11
P3       10       19
P4       18       23

Avg Waiting Time: 8.75
Avg Turnaround Time: 15.25
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 1: FCFS output.

2.2 Exercise 2: SJF, non-preemptive (`sjf.cpp`)

Arrived processes are added to a ready queue, the queue is sorted by burst time, and the shortest one runs to completion. Only P1 is present at $t = 0$, so it runs first; after that the order is P2, P4, P3. Running the short jobs before P3 lowers the average waiting time from 8.75 to 7.75.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/sjf
=== Exercise 2: Shortest Job First (non-preemptive) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Gantt Chart: P1(8) P2(4) P4(5) P3(9)
Timeline:    0 [P1] 8 [P2] 12 [P4] 17 [P3] 26

Process  Waiting  Turnaround
P1       0        8
P2       7        11
P3       15       24
P4       9        14

Avg Waiting Time: 7.75
Avg Turnaround Time: 14.25
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 2: SJF output.

2.3 Exercise 3: SRTF, preemptive (`srtf.cpp`)

The simulation advances one time unit at a time and always runs the arrived process with the least remaining time. P2 preempts P1 at $t = 1$ because its remaining time (4) is shorter than P1's (7). The program logs each preemption, and the Gantt chart starts a new entry at every context switch. SRTF gives the lowest average waiting time of the ten (6.50).

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/srtf
=== Exercise 3: Shortest Remaining Time First (preemptive) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

t=1: P2 (remaining 4) preempts P1 (remaining 7)
Preemptions: 1

Gantt Chart: P1(1) P2(4) P4(5) P1(7) P3(9)
Timeline:    0 [P1] 1 [P2] 5 [P4] 10 [P1] 17 [P3] 26

Process  Waiting  Turnaround
P1       9        17
P2       0        4
P3       15       24
P4       2        7

Avg Waiting Time: 6.5
Avg Turnaround Time: 13
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 3: SRTF output with the preemption logged.

2.4 Exercise 4: Priority with aging (`priority.cpp`)

The ready process with the lowest priority number runs to completion. **Enhancement (aging):** for every 5 time units a process has waited, its priority improves by one level, so a low-priority process cannot starve. The log shows each process's original and aged priority when it is dispatched; by $t = 21$, P4 has aged from priority 4 to 1.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/priority
=== Exercise 4: Priority Scheduling (non-preemptive) with aging ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Scheduling log (aging: +1 priority level per 5 units waited):
t= 0: run P1  (priority 2, aged to 2 after waiting 0)
t= 8: run P2  (priority 1, aged to 1 after waiting 7)
t=12: run P3  (priority 3, aged to 1 after waiting 10)
t=21: run P4  (priority 4, aged to 1 after waiting 18)

Gantt Chart: P1(8) P2(4) P3(9) P4(5)
Timeline:    0 [P1] 8 [P2] 12 [P3] 21 [P4] 26

Process  Waiting  Turnaround
P1       0        8
P2       7        11
P3       10       19
P4       18       23

Avg Waiting Time: 8.75
Avg Turnaround Time: 15.25
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 4: Priority scheduling output with the aging log.

2.5 Exercise 5: Round Robin (`rr.cpp`)

Processes take turns from a FIFO queue with a quantum of 4. A process that still has work left is preempted and moved to the back of the queue; processes that arrived during its time slice are queued ahead of it. Round Robin has a higher average waiting time (11.75) than FCFS here, but every process gets on the CPU much sooner.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/rr
=== Exercise 5: Round Robin (quantum = 4) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Gantt Chart: P1(4) P2(4) P3(4) P4(4) P1(4) P3(4) P4(1) P3(1)
Timeline:    0 [P1] 4 [P2] 8 [P3] 12 [P4] 16 [P1] 20 [P3] 24 [P4] 25 [P3] 26

Process  Waiting  Turnaround
P1       12       20
P2       3        7
P3       15       24
P4       17       22

Avg Waiting Time: 11.75
Avg Turnaround Time: 18.25
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 5: Round Robin output.

2.6 Exercise 6: Multilevel Queue (`mlq.cpp`)

Processes with priority below 3 (P1, P2) go to the high queue, which uses Round Robin with a quantum of 4; the rest (P3, P4) go to the low queue, which is FCFS. The high queue is always served first, and the low queue runs one time unit at a time so a high-priority arrival preempts it immediately.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/mlq
=== Exercise 6: Multilevel Queue (High: RR q=4, Low: FCFS) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

P1 (arrived t=0, priority 2) joins the HIGH queue (RR)
P2 (arrived t=1, priority 1) joins the HIGH queue (RR)
P3 (arrived t=2, priority 3) joins the LOW queue (FCFS)
P4 (arrived t=3, priority 4) joins the LOW queue (FCFS)

Gantt Chart: P1(4) P2(4) P1(4) P3(9) P4(5)
Timeline:    0 [P1] 4 [P2] 8 [P1] 12 [P3] 21 [P4] 26

Process  Waiting  Turnaround
P1       4        12
P2       3        7
P3       10       19
P4       18       23

Avg Waiting Time: 8.75
Avg Turnaround Time: 15.25
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 6: Multilevel Queue output showing the queue assignments.

2.7 Exercise 7: Multilevel Feedback Queue (`mlfq.cpp`)

There are three Round Robin queues with quanta 2, 4 and 8, and every process starts in the top queue. **Feedback:** a process that uses its whole quantum is treated as CPU-bound and demoted one level. **Enhancement:** a process that has waited 8 time units in a lower queue is promoted one level. The feedback log lists every demotion and promotion.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/mlfq
=== Exercise 7: Multilevel Feedback Queue (quanta 2, 4, 8) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Feedback log:
t=2: P1 used its full quantum, demoted Q0 -> Q1
t=4: P2 used its full quantum, demoted Q0 -> Q1
t=6: P3 used its full quantum, demoted Q0 -> Q1
t=8: P4 used its full quantum, demoted Q0 -> Q1
t=12: P1 used its full quantum, demoted Q1 -> Q2
t=12: P2 promoted Q1 -> Q0 (waited 8+ units)
t=14: P3 promoted Q1 -> Q0 (waited 8+ units)
t=16: P3 used its full quantum, demoted Q0 -> Q1
t=16: P4 promoted Q1 -> Q0 (waited 8+ units)
t=18: P4 used its full quantum, demoted Q0 -> Q1
t=22: P3 used its full quantum, demoted Q1 -> Q2
t=22: P1 promoted Q2 -> Q1 (waited 8+ units)

Gantt Chart: P1(2) P2(2) P3(2) P4(2) P1(4) P2(2) P3(2) P4(2) P3(4) P4(1) P1(2) P3(1)
Timeline:    0 [P1] 2 [P2] 4 [P3] 6 [P4] 8 [P1] 12 [P2] 14 [P3] 16 [P4] 18 [P3] 22 [P4] 23 [P1] 25 [P3] 26

Process  Waiting  Turnaround
P1       17       25
P2       9        13
P3       15       24
P4       15       20

Avg Waiting Time: 14
Avg Turnaround Time: 20.5
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 7: MLFQ output with the feedback log.

2.8 Exercise 8: Lottery (`lottery.cpp`)

Each process gets $10/\textit{priority}$ tickets (P1 = 5, P2 = 10, P3 = 3, P4 = 2). At each step a random ticket is drawn among the ready processes and the winner runs for a quantum of 2. **Enhancement (dynamic tickets):** every ready process that loses a draw gains one ticket, and the winner returns to its base amount. The program accepts an optional seed (`./lottery 7`) so that a run can be repeated exactly.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/lottery 7
=== Exercise 8: Lottery Scheduling (quantum = 2) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Random seed: 7
Base tickets: P1=5 P2=10 P3=3 P4=2

Draws:
t= 0: ticket  0 of  5 -> P1  ( P1:5 )
t= 2: ticket  4 of 18 -> P1  ( P1:5 P2:10 P3:3 )
t= 4: ticket 17 of 22 -> P3  ( P1:5 P2:11 P3:4 P4:2 )
t= 6: ticket  7 of 24 -> P2  ( P1:6 P2:12 P3:3 P4:3 )
t= 8: ticket 10 of 25 -> P2  ( P1:7 P2:10 P3:4 P4:4 )
t=10: ticket 17 of 18 -> P4  ( P1:8 P3:5 P4:5 )
t=12: ticket 12 of 17 -> P3  ( P1:9 P3:6 P4:2 )
t=14: ticket  7 of 16 -> P1  ( P1:10 P3:3 P4:3 )
t=16: ticket 12 of 13 -> P4  ( P1:5 P3:4 P4:4 )
t=18: ticket  4 of 13 -> P1  ( P1:6 P3:5 P4:2 )
t=20: ticket  4 of  9 -> P3  ( P3:6 P4:3 )
t=22: ticket  1 of  7 -> P3  ( P3:3 P4:4 )
t=24: ticket  4 of  8 -> P4  ( P3:3 P4:5 )
t=25: ticket  0 of  4 -> P3  ( P3:4 )

Gantt Chart: P1(4) P3(2) P2(4) P4(2) P3(2) P1(2) P4(2) P1(2) P3(4) P4(1) P3(1)
Timeline:    0 [P1] 4 [P3] 6 [P2] 10 [P4] 12 [P3] 14 [P1] 16 [P4] 18 [P1] 20 [P3] 24 [P4] 25 [P3] 26

Process  Waiting  Turnaround
P1       12       20
P2       5        9
P3       15       24
P4       17       22

Avg Waiting Time: 12.25
Avg Turnaround Time: 18.75
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 8: Lottery output showing every draw and the ticket counts.

2.9 Exercise 9: Simplified CFS (`cfs.cpp`)

Ready processes are kept in a min-heap ordered by virtual runtime, and the process with the smallest value runs for a slice of 2. **Enhancement (weights):** each process has a weight of $1024/1.25^{\,priority-1}$ and its virtual runtime grows by $slice \times 1024/weight$, so a higher-priority process accumulates virtual runtime more slowly and receives more CPU time. New arrivals start at the current minimum virtual runtime.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/cfs
=== Exercise 9: Completely Fair Scheduler (simplified, slice = 2) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Weights: P1=819.20 P2=1024.00 P3=655.36 P4=524.29

Final vruntime: P1=10.00 P2=4.00 P3=14.06 P4=9.77

Gantt Chart: P1(2) P2(2) P3(2) P4(2) P2(2) P1(2) P3(2) P4(2) P1(2) P3(2) P1(2) P4(1) P3(3)
Timeline:    0 [P1] 2 [P2] 4 [P3] 6 [P4] 8 [P2] 10 [P1] 12 [P3] 14 [P4] 16 [P1] 18 [P3] 20 [P1] 22 [P4] 23 [P3] 26

Process  Waiting  Turnaround
P1       14       22
P2       5        9
P3       15       24
P4       15       20

Avg Waiting Time: 12.25
Avg Turnaround Time: 18.75
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 9: CFS output with weights and final virtual runtimes.

2.10 Exercise 10: EDF (`edf.cpp`)

Each process gets a deadline of $arrival + 2 \times burst$ (P1 = 16, P2 = 9, P3 = 20, P4 = 13). A priority queue ordered by deadline picks the process to run each time unit, so a nearer deadline preempts. **Enhancement (schedulability):** the program first computes the density $\sum burst/(deadline - arrival)$, which is 2.0 here, so the deadlines are not guaranteed; after the run it reports each deadline as met or missed. P1 and P3 miss theirs.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/edf
=== Exercise 10: Earliest Deadline First (preemptive) ===

Process table:
Process  Arrival  Burst  Priority
P1       0        8      2
P2       1        4      1
P3       2        9      3
P4       3        5      4

Deadlines (arrival + burst * 2): P1=16 P2=9 P3=20 P4=13
Schedulability check: density = 2 > 1, deadlines are NOT guaranteed (overload possible)

t= 5: P2 finished, deadline  9  -> met
t=10: P4 finished, deadline 13  -> met
t=17: P1 finished, deadline 16  -> MISSED
t=26: P3 finished, deadline 20  -> MISSED
Deadlines missed: 2 of 4

Gantt Chart: P1(1) P2(4) P4(5) P1(7) P3(9)
Timeline:    0 [P1] 1 [P2] 5 [P4] 10 [P1] 17 [P3] 26

Process  Waiting  Turnaround
P1       9        17
P2       0        4
P3       15       24
P4       2        7

Avg Waiting Time: 6.5
Avg Turnaround Time: 13
CPU Utilization: 100%
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 10: EDF output with the schedulability check and deadline report.

3 Part 3: Modular Task Scheduling Simulator

3.1 Design

The simulator is built around an abstract base class with one pure virtual method:

```
class Scheduler {
public:
    virtual ~Scheduler() = default;
    virtual void schedule(std::vector<Process>& processes,
                          Gantt& gantt, int& total_time) = 0;
    virtual std::string name() const = 0;
};
```

Each of the ten algorithms is a derived class that overrides `schedule()`. `main()` selects one by name through a small factory function and only ever uses it through a `std::unique_ptr<Scheduler>`.

Adding a scheduler means adding one class and one line in the factory. The base class also provides the helpers every scheduler needs: running a process for a time slice, recording Gantt entries, and inserting idle time.

File	Contents
`simulator.cpp`	`main()`, argument parsing, scheduler factory, reports
`process.h`	the `Process` struct
`scheduler.h`	the abstract `Scheduler` base class and shared helpers
`schedulers.h`	the ten derived scheduler classes
`utils.h`	input file loading, task generation, metrics, printing
`tasks.txt`, `tasks_deadlines.txt`	sample input files

Table 3: Source files of the simulator.

3.2 Usage

```
g++ -std=c++17 -pthread simulator.cpp -o simulator
./simulator --scheduler rr --quantum 4
./simulator --scheduler srtf --input tasks.txt --output log.txt
./simulator --compare --random --num 20 --seed 42
```

Option	Meaning
`--scheduler <name>`	fcfs, sjf, srtf, priority, rr, mlq, mlfq, lottery, cfs, edf
`--compare [a,b,...]`	run several (default: all) schedulers on the same tasks
`--input <file>`	tasks from a file: `ID arrival burst priority [deadline]`
`--random`, `--num <n>`	generate n random tasks (default 10)
`--web`	generate a web-server request workload
`--seed <n>`	seed for random tasks, lottery draws and I/O
`--quantum <n>`	quantum for rr, mlq and lottery (default 4)
`--aging <n>`	priority aging interval (default 5, 0 turns it off)
`--io <percent>`	rr: chance of blocking for I/O after a time slice
`--cpus <n>`	number of CPUs, each simulated in its own thread
`--output <file>`	also write the results to a file

Table 4: Command-line options.

With no task option the simulator uses the assignment table. Besides the Gantt chart it reports average waiting, turnaround and response time, CPU utilization, throughput and the number of context switches.

3.3 Enhancements

- **Priority aging.** A waiting process gains one priority level every `--aging` time units. The scheduler logs each decision that aging changed.
- **EDF deadlines.** Deadlines are read from the optional fifth column of the input file; tasks without one get $arrival + 2 \times burst$. The scheduler reports a density-based schedulability estimate and every missed deadline.

- **I/O simulation (Round Robin).** With `--io p`, a process that has not finished at the end of its time slice blocks for 1–5 time units with probability $p\%$, then rejoins the ready queue. Time spent blocked is not counted as waiting.
- **MLFQ feedback, lottery dynamic tickets, CFS weights.** The same enhancements as in the lab exercises.
- **Web-server scenario and load balancing.** `--web` generates a stream of requests in which about one in five is long. With `--cpus n`, a load balancer sends each request to the CPU that will be free soonest, and each CPU runs its own scheduler instance in a separate `std::thread`.
- **Error handling.** Unknown options and schedulers, missing values, non-numeric or out-of-range numbers and unreadable files produce an error message and a non-zero exit code. Malformed lines in an input file are reported with their line number and skipped.

3.4 Test Runs

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler rr --quantum 4
============================================================
 Task Scheduling Simulator
============================================================
Workload : default assignment table (4 tasks)
CPUs     : 1
Scheduler: Round Robin (quantum = 4)

Gantt Chart: P1(4) P2(4) P3(4) P4(4) P1(4) P3(4) P4(1) P3(1)
Timeline:    0 [P1] 4 [P2] 8 [P3] 12 [P4] 16 [P1] 20 [P3] 24 [P4] 25 [P3] 26

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response
P1             0      8         2      0      20       12          20         0
P2             1      4         1      4       8        3           7         3
P3             2      9         3      8      26       15          24         6
P4             3      5         4     12      25       17          22         9

Average Waiting Time: 11.75
Average Turnaround Time: 18.25
Average Response Time: 4.50
CPU Utilization: 100.00%
Throughput: 0.1538 processes/unit time
Context Switches: 7
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 11: Round Robin on the default tasks (`--scheduler rr --quantum 4`). The result matches the lab program.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler rr --random --seed 42
============================================================
 Task Scheduling Simulator
============================================================
Workload : random tasks, seed 42 (10 tasks)
CPUs     : 1
Scheduler: Round Robin (quantum = 4)

Gantt Chart: P3(4) P1(4) P3(1) P7(4) P1(4) P10(4) P5(4) P6(4) P4(4) P7(3) P2(4) P8(4) P9(4) P10(4) P5(1) P6(2) P4(4) P2(3) P4(1)
Timeline:    0 [P3] 4 [P1] 8 [P3] 9 [P7] 13 [P1] 17 [P10] 21 [P5] 25 [P6] 29 [P4] 33 [P7] 36 [P2] 40 [P8] 44 [P9] 48 [P10] 52 [P5] 53 [P6] 55 [P4] 59 [
P2] 62 [P4] 63

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response
P3             0      5         1      0       9        4           9         0
P1             2      8         2      4      17        7          15         2
P7             7      7         5      9      36       22          29         2
P10            9      8         5     17      52       35          43         8
P5            10      5         2     21      53       38          43        11
P6            12      6         1     25      55       37          43        13
P4            13      9         5     29      63       41          50        16
P2            14      7         1     36      62       41          48        22
P8            16      4         2     40      44       24          28        24
P9            19      4         2     44      48       25          29        25

Average Waiting Time: 27.40
Average Turnaround Time: 33.70
Average Response Time: 12.30
CPU Utilization: 100.00%
Throughput: 0.1587 processes/unit time
Context Switches: 18
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 12: Round Robin on ten random tasks (`--scheduler rr --random --seed 42`).

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler srtf --input submission/Part3_Simulator/tasks.txt
============================================================
 Task Scheduling Simulator
============================================================
Workload : file submission/Part3_Simulator/tasks.txt (12 tasks)
CPUs     : 1
Scheduler: Shortest Remaining Time First (preemptive)

Gantt Chart: T1(1) T2(2) T4(1) T3(2) T6(3) T7(2) T3(1) T9(1) T3(4) T11(2) T12(4) T10(6) T1(9) T8(9) T5(12)
Timeline:    0 [T1] 1 [T2] 3 [T4] 4 [T3] 6 [T6] 9 [T7] 11 [T3] 12 [T9] 13 [T3] 17 [T11] 19 [T12] 23 [T10] 29 [T1] 38 [T8] 47 [T5] 59

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response
T1             0     10         3      0      38       28          38         0
T2             1      2         1      1       3        0           2         0
T3             2      7         4      4      17        8          15         2
T4             3      1         2      3       4        0           1         0
T5             5     12         5     47      59       42          54        42
T6             6      3         1      6       9        0           3         0
T7             8      2         2      9      11        1           3         1
T8            10      9         4     38      47       28          37        28
T9            12      1         1     12      13        0           1         0
T10           14      6         3     23      29        9          15         9
T11           15      2         2     17      19        2           4         2
T12           18      4         5     19      23        1           5         1

Average Waiting Time: 9.92
Average Turnaround Time: 14.83
Average Response Time: 7.08
CPU Utilization: 100.00%
Throughput: 0.2034 processes/unit time
Context Switches: 14
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 13: SRTF on tasks read from `tasks.txt`.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --compare --input submission/Part3_Simulator/tasks.txt --seed 42
============================================================
 Task Scheduling Simulator
============================================================
Workload : file submission/Part3_Simulator/tasks.txt (12 tasks)
CPUs     : 1

Input tasks:
ID       Arrival  Burst  Priority
T1             0     10         3
T2             1      2         1
T3             2      7         4
T4             3      1         2
T5             5     12         5
T6             6      3         1
T7             8      2         2
T8            10      9         4
T9            12      1         1
T10           14      6         3
T11           15      2         2
T12           18      4         5

Comparison (same tasks for every scheduler, quantum = 4):
Scheduler   Avg Wait  Avg Turnaround  Avg Response   CPU Util  Throughput  Switches
--------------------------------------------------------------------------------
fcfs           22.67           27.58         22.67    100.00%      0.2034        11
sjf            12.50           17.42         12.50    100.00%      0.2034        11
srtf            9.92           14.83          7.08    100.00%      0.2034        14
priority       19.83           24.75         19.83    100.00%      0.2034        11
rr             22.17           27.08         12.08    100.00%      0.2034        19
mlq            13.00           17.92         12.08    100.00%      0.2034        15
mlfq           20.25           25.17          3.25    100.00%      0.2034        29
lottery        21.08           26.00         11.67    100.00%      0.2034        19
cfs            18.17           23.08          2.25    100.00%      0.2034        30
edf            10.75           15.67          8.75    100.00%      0.2034        14

Lowest average waiting time : srtf (9.92)
Lowest average response time: cfs (2.25)
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 14: All ten schedulers compared on `tasks.txt`.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler priority --aging 3 --input submission/Part3_Simulator/tasks.txt
============================================================
 Task Scheduling Simulator
============================================================
Workload : file submission/Part3_Simulator/tasks.txt (12 tasks)
CPUs     : 1
Scheduler: Priority (non-preemptive, aging every 3 units)

Gantt Chart: T1(10) T2(2) T3(7) T4(1) T5(12) T6(3) T7(2) T8(9) T9(1) T10(6) T11(2) T12(4)
Timeline:    0 [T1] 10 [T2] 12 [T3] 19 [T4] 20 [T5] 32 [T6] 35 [T7] 37 [T8] 46 [T9] 47 [T10] 53 [T11] 55 [T12] 59

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response
T1             0     10         3      0      10        0          10         0
T2             1      2         1     10      12        9          11         9
T3             2      7         4     12      19       10          17        10
T4             3      1         2     19      20       16          17        16
T5             5     12         5     20      32       15          27        15
T6             6      3         1     32      35       26          29        26
T7             8      2         2     35      37       27          29        27
T8            10      9         4     37      46       27          36        27
T9            12      1         1     46      47       34          35        34
T10           14      6         3     47      53       33          39        33
T11           15      2         2     53      55       38          40        38
T12           18      4         5     55      59       37          41        37

Average Waiting Time: 22.67
Average Turnaround Time: 27.58
Average Response Time: 22.67
CPU Utilization: 100.00%
Throughput: 0.2034 processes/unit time
Context Switches: 11

Scheduler events:
  t=12: aging: T3 (priority 4 aged to 1) runs ahead of T6 (priority 1)
  t=19: aging: T4 (priority 2 aged to 1) runs ahead of T6 (priority 1)
  t=20: aging: T5 (priority 5 aged to 1) runs ahead of T6 (priority 1)
  t=35: aging: T7 (priority 2 aged to 1) runs ahead of T9 (priority 1)
  t=37: aging: T8 (priority 4 aged to 1) runs ahead of T9 (priority 1)
  t=47: aging: T10 (priority 3 aged to 1) runs ahead of T11 (priority 2)
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 15: Priority scheduling with aging every 3 units; the events list shows where aging changed the order.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler edf --input submission/Part3_Simulator/tasks_deadlines.t
xt
============================================================
 Task Scheduling Simulator
============================================================
Workload : file submission/Part3_Simulator/tasks_deadlines.txt (10 tasks)
CPUs     : 1
Scheduler: Earliest Deadline First (preemptive, default deadline = arrival + burst x 2)

Gantt Chart: S1(1) S2(2) S1(1) S4(1) S1(1) S3(2) S6(2) S3(2) S8(2) S5(2) S10(1) S5(3) S7(3) S9(4)
Timeline:    0 [S1] 1 [S2] 3 [S1] 4 [S4] 5 [S1] 6 [S3] 8 [S6] 10 [S3] 12 [S8] 14 [S5] 16 [S10] 17 [S5] 20 [S7] 23 [S9] 27

ID       Arrival  Burst  Priority  Deadline  Start  Finish  Waiting  Turnaround  Response
S1             0      3         2         8      0       6        3           6         0
S2             1      2         1         5      1       3        0           2         0
S3             2      4         3        20      6      12        6          10         4
S4             4      1         1         7      4       5        0           1         0
S5             6      5         2        22     14      20        9          14         8
S6             8      2         1        12      8      10        0           2         0
S7            10      3         3        30     20      23       10          13        10
S8            12      2         2        18     12      14        0           2         0
S9            14      4         4        40     23      27        9          13         9
S10           16      1         1        19     16      17        0           1         0

Average Waiting Time: 3.70
Average Turnaround Time: 6.40
Average Response Time: 3.10
CPU Utilization: 100.00%
Throughput: 0.3704 processes/unit time
Context Switches: 13
Deadlines Missed: 0 of 10

Scheduler events:
  t=0: schedulability: density 3.21 > 1, deadlines are not guaranteed (overload possible)
  t=27: all deadlines met
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 16: EDF with deadlines read from `tasks_deadlines.txt`.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler rr --io 40 --seed 7 --input submission/Part3_Simulator/t
asks.txt
============================================================
 Task Scheduling Simulator
============================================================
Workload : file submission/Part3_Simulator/tasks.txt (12 tasks)
CPUs     : 1
Scheduler: Round Robin (quantum = 4) with I/O simulation (40% block chance)

Gantt Chart: T1(4) T2(2) T3(4) T4(1) T5(4) T6(3) T7(2) T8(4) T1(4) T9(1) T10(4) T11(2) T3(3) T5(4) T12(4) T8(4) T1(2) T10(2) T5(4) T8(1)
Timeline:    0 [T1] 4 [T2] 6 [T3] 10 [T4] 11 [T5] 15 [T6] 18 [T7] 20 [T8] 24 [T1] 28 [T9] 29 [T10] 33 [T11] 35 [T3] 38 [T5] 42 [T12] 46 [T8] 50 [T1] 52
 [T10] 54 [T5] 58 [T8] 59

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response  I/O
T1             0     10         3      0      52       39          52         0    3
T2             1      2         1      4       6        3           5         3    0
T3             2      7         4      6      38       27          36         4    2
T4             3      1         2     10      11        7           8         7    0
T5             5     12         5     11      58       41          53         6    0
T6             6      3         1     15      18        9          12         9    0
T7             8      2         2     18      20       10          12        10    0
T8            10      9         4     20      59       40          49        10    0
T9            12      1         1     28      29       16          17        16    0
T10           14      6         3     29      54       34          40        15    0
T11           15      2         2     33      35       18          20        18    0
T12           18      4         5     42      46       24          28        24    0

Average Waiting Time: 22.33
Average Turnaround Time: 27.67
Average Response Time: 10.17
CPU Utilization: 100.00%
Throughput: 0.2034 processes/unit time
Context Switches: 19

Scheduler events:
  t=4: T1 blocks for I/O (3 units)
  t=10: T3 blocks for I/O (2 units)
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 17: Round Robin with I/O simulation (`--io 40`).

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler rr --web --num 12 --cpus 2 --seed 3
============================================================
 Task Scheduling Simulator
============================================================
Workload : web-server requests, seed 3 (12 tasks)
CPUs     : 2
Scheduler: Round Robin (quantum = 4)

--- CPU 1 ---
Gantt Chart: IDLE(1) REQ1(2) REQ3(2) REQ4(1) REQ5(3) REQ6(1) REQ7(3) REQ9(1) REQ10(14)
Timeline:    0 [IDLE] 1 [REQ1] 3 [REQ3] 5 [REQ4] 6 [REQ5] 9 [REQ6] 10 [REQ7] 13 [REQ9] 14 [REQ10] 28
--- CPU 2 ---
Gantt Chart: IDLE(2) REQ2(4) REQ8(4) REQ2(4) REQ11(3) REQ8(4) REQ12(2) REQ8(2)
Timeline:    0 [IDLE] 2 [REQ2] 6 [REQ8] 10 [REQ2] 14 [REQ11] 17 [REQ8] 21 [REQ12] 23 [REQ8] 25

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response  CPU
REQ1           1      2         1      1       3        0           2         0    1
REQ2           2      8         5      2      14        4          12         0    2
REQ3           3      2         1      3       5        0           2         0    1
REQ4           5      1         2      5       6        0           1         0    1
REQ5           5      3         3      6       9        1           4         1    1
REQ6           5      1         2      9      10        4           5         4    1
REQ7           5      3         1     10      13        5           8         5    1
REQ8           6     10         4      6      25        9          19         0    2
REQ9           7      1         3     13      14        6           7         6    1
REQ10          9     14         3     14      28        5          19         5    1
REQ11         10      3         1     14      17        4           7         4    2
REQ12         11      2         3     21      23       10          12        10    2

Average Waiting Time: 4.00
Average Turnaround Time: 8.17
Average Response Time: 2.92
CPU Utilization: 89.29%
Throughput: 0.4286 processes/unit time
Context Switches: 13
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 18: Web-server workload load-balanced across two CPUs.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler mlfq --output build/log.txt; ls -l build/log.txt
============================================================
 Task Scheduling Simulator
============================================================
Workload : default assignment table (4 tasks)
CPUs     : 1
Scheduler: Multilevel Feedback Queue (quanta 2/4/8, promote after waiting 8)

Gantt Chart: P1(2) P2(2) P3(2) P4(2) P1(4) P2(2) P3(2) P4(2) P3(4) P4(1) P1(2) P3(1)
Timeline:    0 [P1] 2 [P2] 4 [P3] 6 [P4] 8 [P1] 12 [P2] 14 [P3] 16 [P4] 18 [P3] 22 [P4] 23 [P1] 25 [P3] 26

ID       Arrival  Burst  Priority  Start  Finish  Waiting  Turnaround  Response
P1             0      8         2      0      25       17          25         0
P2             1      4         1      2      14        9          13         1
P3             2      9         3      4      26       15          24         2
P4             3      5         4      6      23       15          20         3

Average Waiting Time: 14.00
Average Turnaround Time: 20.50
Average Response Time: 1.50
CPU Utilization: 100.00%
Throughput: 0.1538 processes/unit time
Context Switches: 11

Scheduler events:
  t=2: P1 used its full quantum, demoted Q0 -> Q1
  t=4: P2 used its full quantum, demoted Q0 -> Q1
  t=6: P3 used its full quantum, demoted Q0 -> Q1
  t=8: P4 used its full quantum, demoted Q0 -> Q1
  t=12: P1 used its full quantum, demoted Q1 -> Q2
  t=12: P2 promoted Q1 -> Q0 (waited too long)
  t=14: P3 promoted Q1 -> Q0 (waited too long)
  t=16: P3 used its full quantum, demoted Q0 -> Q1
  t=16: P4 promoted Q1 -> Q0 (waited too long)
  t=18: P4 used its full quantum, demoted Q0 -> Q1
  t=22: P3 used its full quantum, demoted Q1 -> Q2
  t=22: P1 promoted Q2 -> Q1 (waited too long)

Results written to build/log.txt
-rw-r--r-- 1 root root 1668 Oct 13 20:03 build/log.txt
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 19: Writing the results to a file with `--output`.

```
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project# ./build/simulator --scheduler foo; ./build/simulator --scheduler rr --input missing.tx
t; ./build/simulator --scheduler rr --quantum abc
Error: unknown scheduler 'foo'. Choose one of: fcfs sjf srtf priority rr mlq mlfq lottery cfs edf
Error: cannot open input file 'missing.txt'
Error: --quantum needs a whole number from 1 to 1000 (got 'abc')
root@Saint-James:~/Module-4-Assignment-3-Lab-Scheduling-Project#
```

Figure 20: Error handling for an unknown scheduler, a missing file and an invalid number.

3.5 Comparison and Analysis

Table 5 shows all ten schedulers on the 12-task sample workload in `tasks.txt`, which mixes long jobs (bursts of 9–12) with short ones (bursts of 1–3). It was produced with `./simulator --compare --input tasks.txt --seed 42`. CPU utilization (100%) and throughput (0.2034 tasks per time unit) are the same for every scheduler, because on one CPU with no idle time the order of the work does not change when all of it finishes.

Scheduler	Avg waiting	Avg turnaround	Avg response	Context switches
fcfs	22.67	27.58	22.67	11
sjf	12.50	17.42	12.50	11
srtf	9.92	14.83	7.08	14
priority (aging 5)	19.83	24.75	19.83	11
rr ($q = 4$)	22.17	27.08	12.08	19
mlq	13.00	17.92	12.08	15
mlfq	20.25	25.17	3.25	29
lottery (seed 42)	21.08	26.00	11.67	19
cfs	18.17	23.08	2.25	30
edf	10.75	15.67	8.75	14

Table 5: All schedulers on `tasks.txt` (12 tasks, one CPU).

Round Robin vs. SRTF. SRTF has less than half the average waiting time of Round Robin (9.92 against 22.17) and a lower average response time (7.08 against 12.08), with fewer context switches (14 against 19). SRTF wins on these averages because it always finishes the shortest work first. Its weaknesses do not show in the averages: it needs to know each burst length in advance, which a real system can only estimate, and a long job can be postponed indefinitely while shorter ones keep arriving. Round Robin needs no burst estimates and bounds how long any task waits for its next turn, which is why it suits interactive systems, at the cost of more context switches and longer turnaround.

Response time vs. waiting time. MLFQ and CFS have the lowest average response times (3.25 and 2.25) because every new task gets a short slice almost immediately, but they have the highest context-switch counts (29 and 30) and much higher average waiting times than SJF or SRTF, since long jobs are interrupted repeatedly. FCFS is the opposite: the fewest switches, and the worst waiting and response time (22.67), because short tasks queue behind long ones (the convoy effect).

Effect of aging. On the same workload, priority scheduling without aging (`--aging 0`) has an average waiting time of 14.58, but the low-priority task T5 waits 38 time units. With aging every 5 units T5 waits 20, and with aging every 3 units it waits 15, while the average rises to 19.83 and 22.67. Aging trades average waiting time for a bound on how long a low-priority task is postponed.

CPU-bound vs. I/O-bound workloads. For CPU-bound batch work, where only total completion time matters and nobody is waiting on a response, the non-preemptive schedulers are the best fit: SJF gives a low average turnaround (17.42) with the minimum number of context switches, and FCFS is acceptable when the jobs are of similar length. For I/O-bound and interactive work, tasks use the CPU in short bursts and need it again quickly after each I/O operation, so response time is the metric that matters. MLFQ and CFS are the best fit there: a task that gives up the CPU before its quantum ends stays in a high-priority queue (MLFQ) or keeps a low virtual runtime (CFS), so it is scheduled soon after it becomes ready, and CPU-bound tasks sink to the lower queues without being starved. Round Robin is a reasonable simple choice for interactive work but treats both kinds of task the same. In the Round Robin I/O simulation on this workload (`--io 40 --seed 7`), two tasks blocked for I/O and the other tasks used the CPU in the meantime, so utilization stayed at 100%. EDF is the right choice only when tasks have real deadlines: on `tasks_deadlines.txt` it meets all 10.

Load balancing. For the 12-request web workload (`--web --num 12 --seed 3`) under Round Robin, one CPU finishes at $t = 51$ with an average waiting time of 13.25. Two CPUs finish at $t = 28$ with an average waiting time of 4.00, at 89.29% CPU utilization.

4 References

1. Lab handout, *Lab 8: Implementing and Enhancing Task Schedulers in C++* — source of the starter code (`Process` struct, `Scheduler` base class, Round Robin example, argument parsing).

2. A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018. Chapter 5, CPU Scheduling.

3. R. H. Arpaci-Dusseau, A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapters 7–9 (scheduling, MLFQ, lottery scheduling and CFS). `https://pages.cs.wisc.edu/~remzi/OSTEP/`

4. C. A. Waldspurger, W. E. Weihl, "Lottery Scheduling: Flexible Proportional-Share Resource Management," *OSDI*, 1994.

5. C. L. Liu, J. W. Layland, "Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment," *Journal of the ACM*, 20(1), 1973.